

TERRAFORM

Complete Textbook & Reference Guide

Infrastructure as Code — From Beginner to Expert

Tech-Talks

Learning • Growing • Sharing

Table of Contents

1. Introduction to Terraform.....	5
1.1 What is Terraform?	5
1.2 Why Terraform? Benefits and Advantages	5
1.3 Infrastructure as Code (IaC)	6
1.4 Declarative vs Imperative Approaches.....	6
1.5 Common Terraform Use Cases	6
2. Terraform Architecture	7
2.1 Core Components	7
2.2 How Components Interact	7
3. Installation & Setup	9
3.1 Installing Terraform.....	9
On Linux (Ubuntu/Debian)	9
On macOS (using Homebrew)	9
On Windows (using Chocolatey)	9
Verify Installation.....	9
3.2 Version Management.....	9
3.3 Core Setup Commands	10
4. Providers.....	11
4.1 What is a Provider?	11
4.2 Configuring Providers	11
4.3 Multiple Providers & Aliases	11
4.4 Popular Providers	12
5. Resources.....	13
5.1 What are Resources?	13
5.2 Resource Block Structure	13
5.3 Resource Dependencies	13
Implicit Dependencies	13
Explicit Dependencies (depends_on)	13
6. Variables.....	15
6.1 Input Variables.....	15
6.2 Variable Types.....	15
6.3 Variable Files (.tfvars).....	15
6.4 Sensitive Variables	16
7. Outputs	17
7.1 Purpose of Output Values.....	17

7.2 Cross-Module Outputs.....	17
8. Terraform State	18
8.1 What is Terraform State?.....	18
8.2 Local vs Remote State.....	18
8.3 State Locking.....	18
9. Backends	20
9.1 What is a Backend?.....	20
9.2 S3 Backend (AWS).....	20
9.3 Terraform Cloud Backend.....	20
10. Core Terraform Commands	21
10.1 Typical Terraform Workflow.....	21
11. Modules	22
11.1 What are Modules?	22
11.2 Root and Child Modules	22
11.3 Creating and Using Modules.....	22
11.4 Public Module Registry	22
12. Resource Meta-Arguments	24
12.1 depends_on.....	24
12.2 count	24
12.3 for_each	24
12.4 lifecycle Block.....	24
13. Data Sources	26
13.1 What are Data Sources?	26
14. Expressions & Functions.....	27
14.1 Interpolation Syntax.....	27
14.2 Common Built-in Functions.....	27
14.3 Conditional Expressions	28
15. Workspaces	29
15.1 What is a Workspace?.....	29
15.2 Workspace Commands	29
15.3 Using Workspace in Configuration.....	29
16. Terraform Import	30
16.1 Importing Existing Infrastructure	30
16.2 Import Block (Terraform 1.5+).....	30
17. Drift Detection	31
17.1 What is Infrastructure Drift?	31

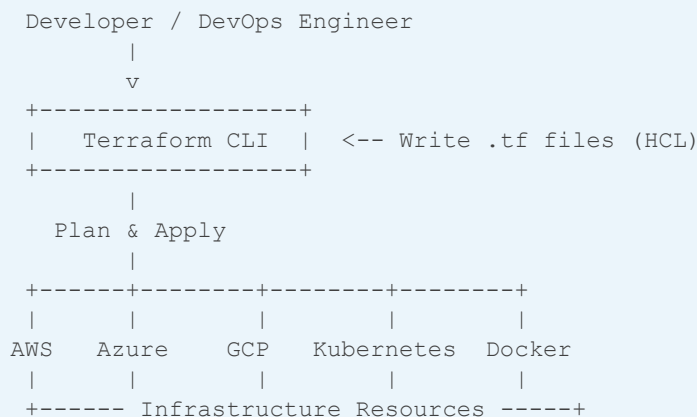
17.2 Detecting and Handling Drift	31
18. Terraform Cloud & Enterprise	32
18.1 Terraform Cloud Features	32
18.2 Sentinel Policy Example	32
19. Security Best Practices	33
19.1 Securing State Files.....	33
19.2 Managing Secrets.....	33
19.3 IAM Best Practices	33
20. Testing & Validation	34
20.1 Built-in Validation Commands.....	34
20.2 Testing Tools.....	34
terraform-compliance	34
Terratest	34
20.3 Terraform Test (Native, v1.6+).....	34
21. Best Practices	36
21.1 Code Organization.....	36
21.2 Naming Conventions	36
21.3 Essential Best Practices Checklist.....	36
22. Comparison with Other Tools.....	37
23. Terraform in DevOps.....	38
23.1 CI/CD Integration.....	38
23.2 GitOps Workflow.....	38
24. Troubleshooting	39
24.1 Debugging Tips	39
25. Real-Time Use Cases	40
25.1 Cloud Infrastructure Provisioning (AWS).....	40
25.2 Multi-Cloud Deployment	40
25.3 Kubernetes Cluster (EKS).....	40
25.4 Network & Security Setup.....	41
25.5 Infrastructure Automation & Scheduling.....	41

1. Introduction to Terraform

1.1 What is Terraform?

Terraform is an open-source Infrastructure as Code (IaC) tool developed by HashiCorp. It enables users to define and provision infrastructure using a high-level, human-readable configuration language called HCL (HashiCorp Configuration Language). Terraform manages resources across a wide variety of service providers — from cloud platforms like AWS, Azure, and GCP to on-premises solutions.

Figure 1.1 — Terraform's Role in Infrastructure Provisioning



1.2 Why Terraform? Benefits and Advantages

- Multi-Cloud Support: Manage AWS, Azure, GCP, and more with a single tool
- Declarative Syntax: Describe the desired end state; Terraform handles the rest
- Idempotency: Running Terraform multiple times produces the same result
- Version Control Friendly: Infrastructure code lives in Git alongside application code
- Huge Provider Ecosystem: 3,000+ providers available in the Terraform Registry
- State Management: Tracks infrastructure state for predictable updates
- Modular & Reusable: Organize code into reusable modules

❏ Quick Tip

Terraform replaces manual click-ops in cloud consoles. Once your infrastructure is defined in code, you can version it, review it, and replicate it in seconds!

1.3 Infrastructure as Code (IaC)

Infrastructure as Code (IaC) is the practice of managing and provisioning computing infrastructure through machine-readable configuration files, rather than through manual processes or interactive configuration tools.

- Consistency: Eliminates configuration drift across environments
- Speed: Provision entire environments in minutes
- Documentation: Your code IS your infrastructure documentation
- Collaboration: Teams can review changes via pull requests
- Disaster Recovery: Recreate environments quickly from code

1.4 Declarative vs Imperative Approaches

Aspect	Declarative (Terraform)	Imperative (Scripts/Ansible)
Approach	Define WHAT you want	Define HOW to do it
Example	<pre>resource "aws_instance" "web" {}</pre>	<pre>aws ec2 run-instances --image-id ...</pre>
State Awareness	Yes — tracks current state	No — may re-run unneeded steps
Idempotency	Built-in	Must be coded manually
Learning Curve	Moderate	Lower initially, complex at scale
Best For	Infrastructure provisioning	Configuration management

1.5 Common Terraform Use Cases

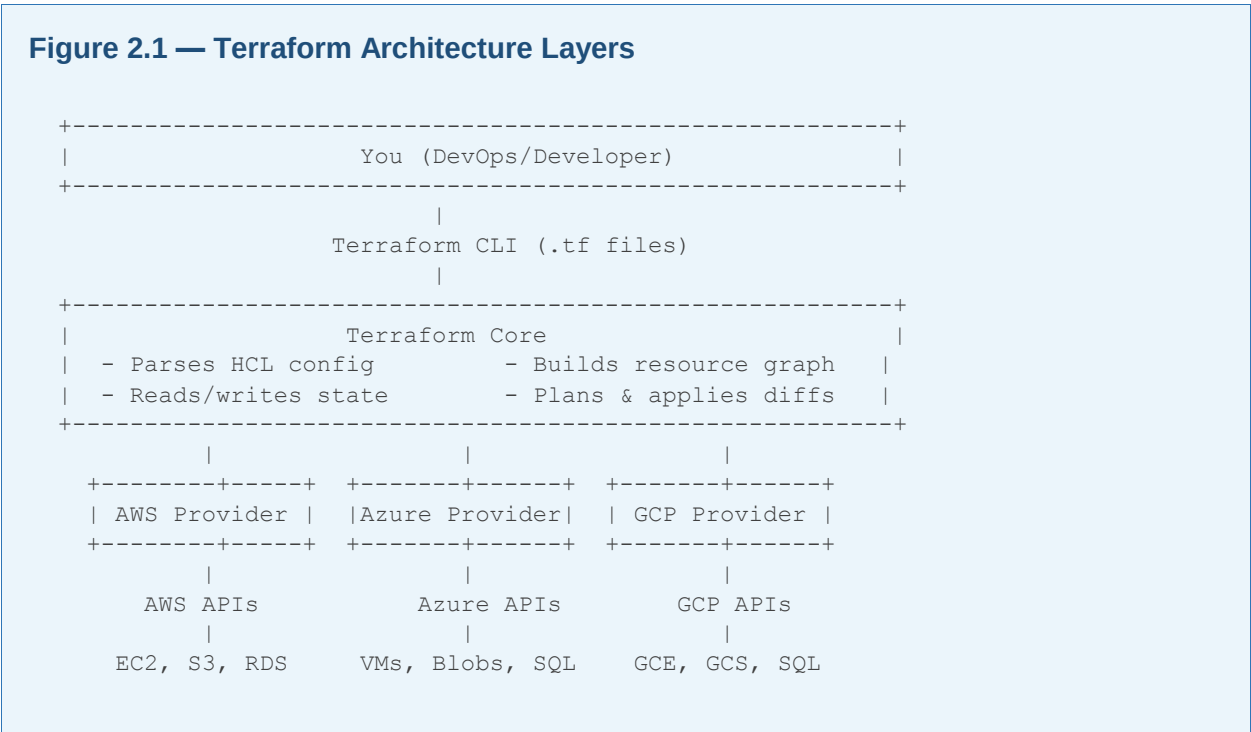
- Cloud infrastructure provisioning (VMs, VPCs, databases)
- Multi-cloud and hybrid cloud deployments
- Kubernetes cluster management
- Network and security group automation
- CI/CD pipeline infrastructure
- SaaS application setup and configuration

2. Terraform Architecture

2.1 Core Components

Component	Description
Terraform CLI	Command-line interface to interact with Terraform. Entry point for all commands.
Terraform Core	Core engine: parses config, builds dependency graph, plans & applies changes.
Providers	Plugins that interact with APIs (AWS, Azure, GCP). Each resource type belongs to a provider.
Resources	The actual infrastructure objects managed (e.g., aws_instance, azurerm_vm).
State File	JSON file tracking the current state of managed infrastructure (terraform.tfstate).
Modules	Reusable groups of resources packaged together for easier consumption.

Figure 2.1 — Terraform Architecture Layers



2.2 How Components Interact

1. You write .tf configuration files describing desired infrastructure.

2. Terraform Core reads configs and the existing state file.
3. Core builds a dependency graph of resources.
4. Provider plugins translate resource definitions into API calls.
5. APIs provision actual infrastructure objects.
6. Terraform updates the state file to reflect the new actual state.

3. Installation & Setup

3.1 Installing Terraform

On Linux (Ubuntu/Debian)

```
# Add HashiCorp GPG key
wget -O- https://apt.releases.hashicorp.com/gpg | gpg --dearmor | sudo tee
/usr/share/keyrings/hashicorp-archive-keyring.gpg

# Add HashiCorp repository
echo "deb [signed-by=/usr/share/keyrings/hashicorp-archive-keyring.gpg]
https://apt.releases.hashicorp.com $(lsb_release -cs) main" | sudo tee
/etc/apt/sources.list.d/hashicorp.list

# Install Terraform
sudo apt-get update && sudo apt-get install terraform
```

On macOS (using Homebrew)

```
brew tap hashicorp/tap
brew install hashicorp/tap/terraform
```

On Windows (using Chocolatey)

```
choco install terraform
```

Verify Installation

```
terraform version
# Output: Terraform v1.7.0 on linux_amd64
```

3.2 Version Management

Use tfenv (Terraform Version Manager) to switch between versions easily — critical for teams working on different projects.

```
# Install tfenv
git clone https://github.com/tfutils/tfenv.git ~/.tfenv

# Install a specific version
tfenv install 1.6.6
tfenv use 1.6.6
```

□ Version Pinning Tip

Always pin the Terraform version in your project using a `.terraform-version` file or `required_version = ">= 1.5.0"` in your `terraform {}` block. This prevents breaking changes from unexpected upgrades.

3.3 Core Setup Commands

Command	Purpose	When to Use
<code>terraform init</code>	Initializes the working directory. Downloads providers & modules.	First time and after adding new providers/modules
<code>terraform fmt</code>	Formats <code>.tf</code> files to canonical style.	Before committing code to version control
<code>terraform validate</code>	Validates syntax and configuration logic.	After writing or editing configuration files

✔ Best Practice

Run `terraform fmt` and `terraform validate` as pre-commit hooks to enforce code quality standards across your team automatically.

4. Providers

4.1 What is a Provider?

A Provider is a plugin that Terraform uses to interact with external APIs. Providers implement resource types and data sources. Without a provider, Terraform cannot manage any infrastructure. Each provider is responsible for understanding API interactions and exposing resources.

4.2 Configuring Providers

```
# Basic AWS Provider configuration
terraform {
  required_providers {
    aws = {
      source  = "hashicorp/aws"
      version = "~> 5.0"
    }
  }
}

provider "aws" {
  region = "us-east-1"
}
```

4.3 Multiple Providers & Aliases

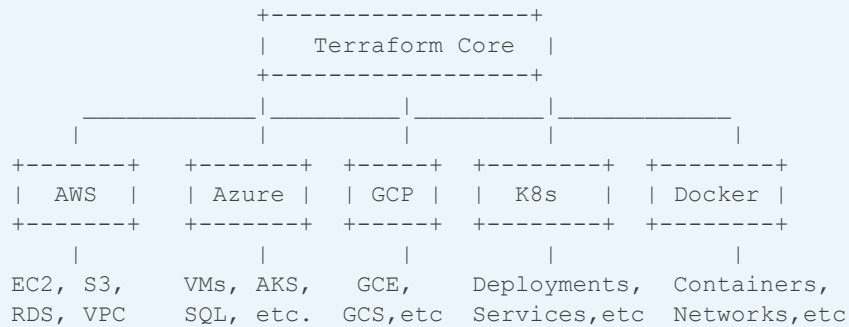
```
# Multiple AWS regions using aliases
provider "aws" {
  alias  = "us_east"
  region = "us-east-1"
}

provider "aws" {
  alias  = "eu_west"
  region = "eu-west-1"
}

resource "aws_instance" "app_us" {
  provider = aws.us_east
  ami      = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"
}
```

4.4 Popular Providers

Figure 4.1 — Providers Connecting Terraform to Different Platforms



Provider	Source	Key Resources
AWS	hashicorp/aws	aws_instance, aws_s3_bucket, aws_vpc, aws_rds_cluster
Azure	hashicorp/azurerm	azurerm_virtual_machine, azurerm_storage_account
GCP	hashicorp/google	google_compute_instance, google_storage_bucket
Kubernetes	hashicorp/kubernetes	kubernetes_deployment, kubernetes_service
Docker	kreuzwerker/docker	docker_container, docker_image, docker_network

5. Resources

5.1 What are Resources?

Resources are the most important element in Terraform. Each resource block describes one or more infrastructure objects, such as virtual machines, DNS records, or databases. Resources are defined using the resource block and are the primary building blocks of your infrastructure.

5.2 Resource Block Structure

```
# Syntax: resource "<PROVIDER_TYPE>" "<LOCAL_NAME>" {
#   argument = value
# }

resource "aws_instance" "web_server" {
  ami           = "ami-0c55b159cbfaffe1f0" # Required argument
  instance_type = "t2.micro"                # Required argument

  tags = {
    Name        = "Web Server"
    Environment = "production"
  }
}
```

5.3 Resource Dependencies

Implicit Dependencies

Terraform automatically identifies dependencies when one resource references another using interpolation.

```
resource "aws_vpc" "main" {
  cidr_block = "10.0.0.0/16"
}

resource "aws_subnet" "public" {
  vpc_id      = aws_vpc.main.id # Implicit dependency!
  cidr_block = "10.0.1.0/24"
}
```

Explicit Dependencies (depends_on)

```
resource "aws_instance" "app" {
  ami           = "ami-0c55b159cbfaffe1f0"
  instance_type = "t2.micro"
  depends_on = [aws_vpc.main]
```

```
depends_on = [aws_iam_role_policy.app_policy] # Explicit dependency
}
```

□ **Important Note**

Use explicit `depends_on` sparingly. Terraform's implicit dependency detection is usually sufficient. Overusing `depends_on` can slow down applies and obscure logic.

6. Variables

6.1 Input Variables

Input variables make Terraform configurations flexible and reusable. They serve as parameters to a module or configuration, allowing values to be set externally.

```
variable "instance_type" {
  description = "EC2 instance type"
  type        = string
  default     = "t2.micro"
}

variable "environment" {
  description = "Deployment environment"
  type        = string
  # No default = required input
}

# Usage in resource:
resource "aws_instance" "web" {
  instance_type = var.instance_type
}
```

6.2 Variable Types

Type	Example	Description
string	"t2.micro"	A sequence of Unicode characters
number	42 or 3.14	Integer or floating point number
bool	true or false	Boolean true or false
list(string)	["us-east-1a", "us-east-1b"]	Ordered sequence of same-type values
map(string)	{ Name = "web", Env = "prod" }	Key-value pairs with string values
object({...})	{ name = string, count = number }	Structured type with named attributes
any	any value	Type constraint is skipped

6.3 Variable Files (.tfvars)

```
# File: terraform.tfvars
instance_type = "t3.medium"
environment   = "production"
region        = "us-east-1"
```

```
# Run with:
terraform apply -var-file="production.tfvars"
```

6.4 Sensitive Variables

```
variable "db_password" {
  description = "Database password"
  type        = string
  sensitive   = true    # Prevents value from appearing in logs/plan output
}
```

✔ Best Practice

Never hard-code secrets in .tf files. Use sensitive = true for variables containing secrets, and store actual values in a secrets manager like AWS Secrets Manager, HashiCorp Vault, or pass them via environment variables (TF_VAR_name).

7. Outputs

7.1 Purpose of Output Values

Output values make information about your infrastructure available on the command line, and can expose information for other Terraform configurations to use. They are like return values from a Terraform module.

```
output "instance_ip" {
  description = "Public IP of the web server"
  value      = aws_instance.web.public_ip
}

output "db_endpoint" {
  description = "Database connection endpoint"
  value      = aws_db_instance.main.endpoint
  sensitive  = true    # Will not display in terminal
}
```

7.2 Cross-Module Outputs

```
# In child module (modules/networking/outputs.tf)
output "vpc_id" {
  value = aws_vpc.main.id
}

# In root module (main.tf)
module "networking" {
  source = "../modules/networking"
}

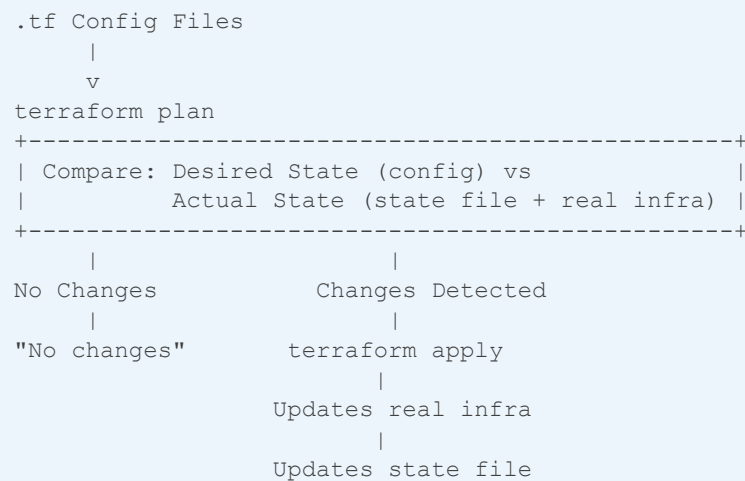
# Reference child module output
resource "aws_instance" "app" {
  subnet_id = module.networking.vpc_id
}
```

8. Terraform State

8.1 What is Terraform State?

Terraform uses a state file (terraform.tfstate) to map real-world resources to your configuration, track metadata, and improve performance for large infrastructures. The state file is the source of truth about what Terraform manages.

Figure 8.1 — State Management Workflow



8.2 Local vs Remote State

Aspect	Local State	Remote State
Storage	Local disk (terraform.tfstate)	S3, GCS, Azure Blob, Terraform Cloud
Team Access	Single user only	Shared across team
State Locking	No locking	Prevents concurrent applies
Security	Plain text, no encryption	Encrypted at rest & in transit
Backup	Manual	Automatic versioning
Best For	Development/testing only	Production & team environments

8.3 State Locking

State locking prevents multiple team members from running Terraform simultaneously, which could corrupt the state file. Remote backends like S3+DynamoDB and Terraform Cloud support automatic locking.

✓ **Best Practice**

Always use remote state with locking enabled in team environments.

Use S3 + DynamoDB for AWS, or Terraform Cloud for a managed solution.

Never manually edit the state file — use terraform state commands instead.

9. Backends

9.1 What is a Backend?

A backend defines where Terraform stores its state data. By default, Terraform uses a local backend, storing state in a local file. Remote backends store state remotely and enable team collaboration with features like state locking.

9.2 S3 Backend (AWS)

```
terraform {
  backend "s3" {
    bucket      = "my-terraform-state-bucket"
    key         = "prod/terraform.tfstate"
    region     = "us-east-1"
    encrypt     = true
    dynamodb_table = "terraform-state-lock" # For state locking
  }
}
```

9.3 Terraform Cloud Backend

```
terraform {
  cloud {
    organization = "my-org"
    workspaces {
      name = "production"
    }
  }
}
```

□ Remote Backend Benefits

Remote backends provide: team collaboration, state locking, encryption, versioning, audit logs, and automated backups. Always prefer remote backends over local for any production or shared environment.

10. Core Terraform Commands

Command	Description	Common Options
terraform init	Initialize working directory. Downloads providers, modules, configures backend.	-upgrade, -reconfigure
terraform plan	Preview changes Terraform will make. Compares config with state.	-out=plan.tfplan, -var-file=...
terraform apply	Apply the planned changes to provision/modify infrastructure.	-auto-approve, -target=resource
terraform destroy	Destroy all managed infrastructure. Use with caution!	-auto-approve, -target=resource
terraform show	Show human-readable output of state or a saved plan.	terraform show plan.tfplan
terraform state	Advanced state management subcommands.	list, show, mv, rm, pull, push
terraform output	Display output values from state.	-json for machine-readable
terraform import	Import existing infrastructure into Terraform state.	See Chapter 17
terraform workspace	Manage workspaces (environments).	new, select, list, delete
terraform fmt	Format HCL code to canonical style.	-recursive for all subdirs
terraform validate	Validate configuration syntax and logic.	No cloud calls made
terraform refresh	Sync state with real-world infrastructure.	Use plan -refresh-only instead
terraform taint	Mark resource for recreation on next apply (deprecated in v0.15+).	Use -replace flag instead

10.1 Typical Terraform Workflow

7. terraform init — initialize directory
8. terraform fmt — format code
9. terraform validate — validate syntax
10. terraform plan — preview changes
11. terraform apply — provision infrastructure
12. terraform destroy — clean up when done

11. Modules

11.1 What are Modules?

Modules are containers for multiple resources used together. They are the primary way to package and reuse resource configurations in Terraform. Every Terraform configuration has at least one module — the root module.

11.2 Root and Child Modules

Type	Description
Root Module	The main working directory where you run terraform commands. Contains main.tf, variables.tf, outputs.tf.
Child Module	A module called by another module using the module block. Can be local (./modules/networking) or from the registry.

11.3 Creating and Using Modules

```
# Directory Structure:
# .
# ├── main.tf
# ├── variables.tf
# ├── outputs.tf
# └── modules/
#     └── ec2/
#         ├── main.tf
#         ├── variables.tf
#         └── outputs.tf

# Root main.tf - calling the module
module "web_server" {
  source      = "./modules/ec2"
  instance_type = "t3.medium"
  environment = "production"
}

# Access module outputs
output "web_ip" {
  value = module.web_server.public_ip
}
```

11.4 Public Module Registry

```
# Using a module from the Terraform Registry
module "vpc" {
  source  = "terraform-aws-modules/vpc/aws"
  version = "5.0.0"

  name = "my-vpc"
  cidr = "10.0.0.0/16"

  azs          = ["us-east-1a", "us-east-1b"]
  private_subnets = ["10.0.1.0/24", "10.0.2.0/24"]
  public_subnets  = ["10.0.101.0/24", "10.0.102.0/24"]
}
```

12. Resource Meta-Arguments

12.1 depends_on

Forces Terraform to wait for a resource to be created before creating another, even when no implicit reference exists.

```
resource "aws_instance" "app" {
  ami          = var.ami_id
  instance_type = "t2.micro"
  depends_on   = [aws_iam_role_policy.app_policy]
}
```

12.2 count

Creates multiple instances of a resource from a single block.

```
resource "aws_instance" "web" {
  count      = 3    # Creates 3 instances
  ami       = var.ami_id
  instance_type = "t2.micro"
  tags = {
    Name = "web-server-${count.index}" # 0, 1, 2
  }
}

# Reference: aws_instance.web[0], aws_instance.web[1], ...
```

12.3 for_each

Creates one instance per item in a map or set. Better than count for named resources.

```
resource "aws_s3_bucket" "data" {
  for_each = toset(["logs", "assets", "backups"])
  bucket   = "company-${each.key}"
}

# Reference: aws_s3_bucket.data["logs"], aws_s3_bucket.data["assets"]
```

12.4 lifecycle Block

```
resource "aws_instance" "web" {
  ami          = var.ami_id
  instance_type = "t3.medium"

  lifecycle {
    create_before_destroy = true # Create new before deleting old (zero downtime)
  }
}
```



```
    prevent_destroy    = true    # Block terraform destroy for this resource
    ignore_changes     = [tags]  # Ignore external changes to tags
  }
}
```

Lifecycle Setting	Effect	Common Use Case
create_before_destroy	New resource created before old one deleted	Zero-downtime deployments
prevent_destroy	Blocks accidental deletion	Production databases, critical resources
ignore_changes	Ignores specified attribute changes	Tags managed outside Terraform
replace_triggered_by	Forces replacement when a reference changes	When config changes need new resources

13. Data Sources

13.1 What are Data Sources?

Data sources allow Terraform to fetch information defined outside of Terraform, or defined by another separate Terraform configuration. Unlike resources, data sources only read information — they do not create or modify infrastructure.

```
# Fetch the latest Amazon Linux 2 AMI
data "aws_ami" "amazon_linux" {
  most_recent = true
  owners      = ["amazon"]

  filter {
    name     = "name"
    values   = ["amzn2-ami-hvm-*-x86_64-gp2"]
  }
}

# Use the data source
resource "aws_instance" "web" {
  ami           = data.aws_ami.amazon_linux.id
  instance_type = "t2.micro"
}
```

Aspect	Resources	Data Sources
Purpose	Create/manage infrastructure	Read existing infrastructure
Syntax	resource "type" "name" {}	data "type" "name" {}
Reference	aws_instance.web.id	data.aws_ami.linux.id
Side Effects	Creates/modifies/deletes	Read-only, no changes

14. Expressions & Functions

14.1 Interpolation Syntax

```
# Reference a resource attribute
value = aws_instance.web.public_ip

# Reference a variable
value = var.environment

# String interpolation
name = "server-${var.environment}-${count.index}"

# Local values
locals {
  common_tags = {
    Project      = var.project_name
    Environment   = var.environment
    ManagedBy    = "Terraform"
  }
}
```

14.2 Common Built-in Functions

Function	Example	Result
length()	length(["a", "b", "c"])	3
join()	join(",", ["a", "b", "c"])	"a,b,c"
split()	split(",", "a,b,c")	["a", "b", "c"]
lookup()	lookup({dev="t2", prod="t3"}, "prod", "t2")	"t3"
element()	element(["a", "b", "c"], 1)	"b"
toset()	toset(["a", "b", "a"])	{"a", "b"} (deduplicated)
flatten()	flatten([[1,2],[3,4]])	[1,2,3,4]
merge()	merge({a=1},{b=2})	{a=1, b=2}
upper()	upper("hello")	"HELLO"
format()	format("server-%02d", 5)	"server-05"
file()	file("userdata.sh")	Contents of the file
jsonencode()	jsonencode({key="value"})	{"key":"value"}

14.3 Conditional Expressions

```
# Syntax: condition ? true_value : false_value

variable "is_production" {
  default = false
}

resource "aws_instance" "web" {
  instance_type = var.is_production ? "t3.large" : "t2.micro"
}
```

15. Workspaces

15.1 What is a Workspace?

Workspaces allow you to manage multiple distinct sets of infrastructure using the same configuration. Each workspace has its own state file, allowing you to use the same Terraform code for different environments (dev, staging, production).

15.2 Workspace Commands

```
# List workspaces (default is always present)
terraform workspace list

# Create a new workspace
terraform workspace new staging

# Switch workspace
terraform workspace select production

# Show current workspace
terraform workspace show

# Delete a workspace
terraform workspace delete staging
```

15.3 Using Workspace in Configuration

```
resource "aws_instance" "web" {
  instance_type = terraform.workspace == "production" ? "t3.large" : "t2.micro"

  tags = {
    Environment = terraform.workspace
  }
}
```

❏ Important Note

Workspaces are not a replacement for separate environments in production. For critical environments, use separate Terraform state files/backends per environment for stronger isolation and access control.

16. Terraform Import

16.1 Importing Existing Infrastructure

Terraform import allows you to bring existing cloud resources under Terraform management without recreating them. This is essential when migrating from manually created infrastructure to IaC.

```
# Step 1: Write the resource block in your .tf file
resource "aws_instance" "my_existing_server" {
  # Leave arguments empty initially
}

# Step 2: Run import command
terraform import aws_instance.my_existing_server i-1234567890abcdef0

# Step 3: Run terraform show to see the current state
terraform show

# Step 4: Fill in the .tf file with the actual values
# Step 5: Run terraform plan to verify no unintended changes
```

16.2 Import Block (Terraform 1.5+)

```
# New declarative import (Terraform >= 1.5)
import {
  to = aws_instance.my_server
  id = "i-1234567890abcdef0"
}

# Terraform will generate config with: terraform plan -generate-config-
out=generated.tf
```

❑ Import Limitations

Not all resources support import. Check the provider documentation for each resource type. After import, always run `terraform plan` to verify the config matches reality before making any changes.

17. Drift Detection

17.1 What is Infrastructure Drift?

Drift occurs when the actual state of infrastructure diverges from what Terraform expects (the state file). This can happen due to manual changes in the cloud console, other tools modifying resources, or automatic updates by cloud services.

17.2 Detecting and Handling Drift

```
# Refresh state to sync with real infrastructure
terraform refresh    # deprecated in newer versions

# Preferred: plan with refresh
terraform plan -refresh-only

# Apply the refresh to update state
terraform apply -refresh-only

# Full plan also detects drift
terraform plan    # shows differences between state and real infra
```

✓ Best Practice

Run `terraform plan -refresh-only` regularly (e.g. in a scheduled CI/CD job) to detect and alert on infrastructure drift. Never let drift accumulate — it leads to unpredictable behavior and security vulnerabilities.

18. Terraform Cloud & Enterprise

18.1 Terraform Cloud Features

Feature	Description
Remote Runs	Plan and apply run in Terraform Cloud — not on developer machines
State Management	Automatic remote state with versioning, locking, and access control
Sentinel Policies	Policy-as-code framework to enforce governance and compliance rules
Team Collaboration	Role-based access, audit logs, SSO integration
Private Registry	Host and share private modules across your organization
Variable Management	Securely store and share variables/secrets across workspaces
Cost Estimation	Preview infrastructure costs before applying changes
VCS Integration	Auto trigger runs on Git commits/pull requests

18.2 Sentinel Policy Example

```
# Sentinel policy: Require tags on all AWS resources
import "tfplan/v2" as tfplan

required_tags = ["Environment", "Owner", "Project"]

all_resources_have_tags = rule {
  all tfplan.resource_changes as _, resource {
    all required_tags as tag {
      resource.change.after.tags contains tag
    }
  }
}

main = rule { all_resources_have_tags }
```


19. Security Best Practices

19.1 Securing State Files

- Enable encryption at rest for remote state (S3 with SSE-S3 or SSE-KMS)
- Use private S3 buckets — never public
- Enable versioning on the state bucket for rollback capability
- Restrict access to state using IAM policies
- Never commit terraform.tfstate to version control

19.2 Managing Secrets

```
# Use environment variables for secrets
export TF_VAR_db_password="supersecret"

# Or use AWS Secrets Manager via data source
data "aws_secretsmanager_secret_version" "db_creds" {
  secret_id = "prod/db/credentials"
}

locals {
  db_password =
jsondecode(data.aws_secretsmanager_secret_version.db_creds.secret_string)["password"]
}
```

19.3 IAM Best Practices

- Use least-privilege IAM roles for Terraform execution
- Use IAM roles instead of static access keys where possible
- Enable MFA for human users with AWS access
- Use separate IAM roles per environment (dev, staging, prod)
- Regularly audit and rotate credentials

20. Testing & Validation

20.1 Built-in Validation Commands

Command	What it Checks
terraform validate	HCL syntax, required arguments, valid values, type checking
terraform plan	Config vs state differences, resource dependency order, API calls for validation
terraform fmt	Code formatting (idempotent — can be automated)

20.2 Testing Tools

terraform-compliance

BDD-style testing framework for Terraform — define policies using Given/When/Then syntax.

```
# Scenario: Ensure S3 buckets have versioning enabled
Scenario: S3 Buckets must have versioning enabled
  Given I have aws_s3_bucket resource configured
  When its versioning is configured
  Then it must contain enabled
  And its value must be true
```

Terratest

A Go library for automated testing of Terraform code by actually deploying real infrastructure.

```
// Example Terratest in Go
func TestTerraformAwsExample(t *testing.T) {
    opts := &terraform.Options{
        TerraformDir: "../examples/terraform-aws-example",
        Vars: map[string]interface{}{ "instance_type": "t2.micro" },
    }
    defer terraform.Destroy(t, opts)
    terraform.InitAndApply(t, opts)
    ip := terraform.Output(t, opts, "public_ip")
    http_helper.HttpGetWithRetry(t, "http://"+ip, 200, "Hello, World", 10,
    5*time.Second)
}
```

20.3 Terraform Test (Native, v1.6+)

```
# test/main.tftest.hcl
run "verify_instance_type" {
    command = plan
    variables {
```

```
    instance_type = "t2.micro"
  }
  assert {
    condition     = aws_instance.web.instance_type == "t2.micro"
    error_message = "Instance type must be t2.micro"
  }
}
```

21. Best Practices

21.1 Code Organization

File	Purpose
main.tf	Primary resource definitions
variables.tf	Input variable declarations
outputs.tf	Output value definitions
providers.tf	Provider and backend configuration
locals.tf	Local values and computed expressions
data.tf	Data source definitions
terraform.tfvars	Variable values (gitignore sensitive ones!)

21.2 Naming Conventions

- Use underscores for resource names: `aws_instance.web_server` (not `web-server`)
- Use descriptive, environment-aware names: `prod_web_sg` vs just `sg`
- Prefix modules by function: `networking`, `compute`, `database`, `security`
- Include environment and project in tags: `Environment=prod`, `Project=payments`

21.3 Essential Best Practices Checklist

✓ Best Practice

1. Always use remote state with locking — never local state in teams
2. Pin provider versions: `version = "~> 5.0"` (tilde-range)
3. Pin Terraform version: `required_version = ">= 1.6.0"`
4. Use modules for reusable patterns — DRY (Don't Repeat Yourself)
5. Separate state per environment (dev/staging/prod workspaces or backends)
6. Use terraform plan output files: `terraform plan -out=tfplan` before apply
7. Review plan output carefully before applying, especially for destroy actions
8. Never store secrets in `.tf` files — use variables + secrets managers
9. Add `.gitignore`: `*.tfstate`, `*.tfstate.backup`, `.terraform/`, `*.tfvars` (if sensitive)
10. Use consistent tagging: `Environment`, `Project`, `Owner`, `ManagedBy=Terraform`

22. Comparison with Other Tools

Tool	Type	Approach	Best For	Language
Terraform	IaC	Declarative	Multi-cloud provisioning	HCL
Ansible	Config Mgmt	Imperative/Declarative	Config management, app deploy	YAML
CloudFormation	IaC	Declarative	AWS-only infrastructure	YAML/JSON
ARM Templates	IaC	Declarative	Azure-only infrastructure	JSON/Bicep
Pulumi	IaC	Declarative	IaC with real programming languages	Python/JS/Go
Chef/Puppet	Config Mgmt	Declarative	Server configuration mgmt	Ruby DSL

□ Terraform vs Ansible

Use Terraform for provisioning infrastructure (VMs, networks, databases) and Ansible for configuring servers (installing packages, managing files, services). They complement each other beautifully in a DevOps pipeline!

23. Terraform in DevOps

23.1 CI/CD Integration

```
# GitHub Actions example: Terraform CI/CD
name: Terraform
on: [push, pull_request]

jobs:
  terraform:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v3
      - uses: hashicorp/setup-terraform@v3
        with:
          terraform_version: 1.7.0
      - run: terraform init
      - run: terraform fmt -check
      - run: terraform validate
      - run: terraform plan -out=tfplan
      # Apply only on main branch push
      - run: terraform apply tfplan
        if: github.ref == "refs/heads/main"
```

23.2 GitOps Workflow

- Feature branch: Developer writes Terraform code
- Pull Request: CI runs terraform plan, posts output as PR comment
- Code Review: Team reviews both code and plan output
- Merge to main: CI/CD automatically runs terraform apply
- Audit: All changes tracked in Git history

24. Troubleshooting

Issue	Cause	Solution
Error: Failed to install provider	Network issue or version constraint conflict	Check internet, try terraform init -upgrade
State lock error	Another process left a lock (crashed)	Delete lock from DynamoDB or use -lock=false carefully
Provider authentication error	Missing/wrong credentials	Check AWS_ACCESS_KEY_ID, AWS_SECRET_ACCESS_KEY env vars
Resource already exists	Resource exists outside Terraform state	Use terraform import to bring under management
Dependency cycle error	Circular resource references	Refactor dependencies to remove cycles
Plan shows unexpected destroy	State drift or config change	Review changes carefully, use targeted apply if needed
Error acquiring state lock	Stale DynamoDB lock record	Manually delete lock item from DynamoDB table
Module not found	Wrong source path or registry URL	Check module source path/URL, run terraform init

24.1 Debugging Tips

```
# Enable detailed logging
export TF_LOG=DEBUG      # Options: TRACE, DEBUG, INFO, WARN, ERROR
export TF_LOG_PATH=terraform.log

# Target specific resource
terraform apply -target=aws_instance.web

# Unlock state manually
terraform force-unlock LOCK_ID
```

25. Real-Time Use Cases

25.1 Cloud Infrastructure Provisioning (AWS)

```
# Complete web application stack
module "vpc"    { source = "./modules/vpc" }
module "ec2"    { source = "./modules/ec2"; vpc_id = module.vpc.vpc_id }
module "rds"    { source = "./modules/rds"; vpc_id = module.vpc.vpc_id }
module "alb"    { source = "./modules/alb"; vpc_id = module.vpc.vpc_id }
```

25.2 Multi-Cloud Deployment

```
# Deploy identical app to AWS and GCP
provider "aws"    { region = "us-east-1" }
provider "google" { project = "my-project"; region = "us-central1" }

module "aws_app" {
  source      = "./modules/app"
  providers = { cloud = aws }
}

module "gcp_app" {
  source      = "./modules/app"
  providers = { cloud = google }
}
```

25.3 Kubernetes Cluster (EKS)

```
module "eks" {
  source          = "terraform-aws-modules/eks/aws"
  cluster_name    = "prod-cluster"
  cluster_version = "1.28"

  vpc_id      = module.vpc.vpc_id
  subnet_ids  = module.vpc.private_subnets

  eks_managed_node_groups = {
    general = {
      desired_size = 2
      min_size     = 1
      max_size     = 5
      instance_types = ["t3.medium"]
    }
  }
}
```


25.4 Network & Security Setup

```
# Security Group with strict rules
resource "aws_security_group" "web" {
  name     = "web-sg"
  vpc_id   = aws_vpc.main.id

  ingress {
    from_port = 443
    to_port   = 443
    protocol  = "tcp"
    cidr_blocks = ["0.0.0.0/0"]
  }

  egress {
    from_port = 0
    to_port   = 0
    protocol  = "-1"
    cidr_blocks = ["0.0.0.0/0"]
  }
}
```

25.5 Infrastructure Automation & Scheduling

Terraform combined with CI/CD pipelines enables automated infrastructure management: nightly drift detection runs, auto-scaling configuration updates, cost optimization through scheduled resource scaling, and self-healing infrastructure that automatically corrects drift.

□ Key Takeaway

Terraform shines when you need repeatable, version-controlled, multi-cloud infrastructure. Start small — provision one resource, master the workflow, then expand to full environments. The investment in IaC pays off enormously as your infrastructure grows!